

Predict-then-Optimize Traffic Engineering for the 5G User Plane

Forecast-driven UPF load balancing, predictive scale-out, and receding-horizon control

Bharadwaj Amrutur

Indian Institute of Science, Bangalore

Version 0.1 · July 2026 · Licensed CC BY-NC 4.0

Abstract

In a 5G core the user-plane function (UPF) carries every user packet, yet the session-management function (SMF) selects a UPF per session **locally and myopically** — so a skewed hash or a hot slice can drive one anchor into overload while peers idle. We present a **predict-then-optimize** control system that forecasts per-class user-plane demand and rebalances (and scales) UPF capacity **ahead** of overload rather than reacting after it. The optimizer is a **min-max linear program** that minimizes the worst UPF Overload Indicator across a cluster; formulated over the epigraph it stays convex and solver-agnostic, so the same model runs on a CPU LP solver for a metro cluster and a GPU interior-point/PDLP backend for a coupled operator-scale twin — a one-line change that is measured to solve a 384,001-variable metro program $\sim 8\times$ faster on GPU. A composable forecasting engine supplies the demand: a calendar-aware log-linear exogenous regression over a small teletraffic generative model, wrapped by ensembling and adaptive-conformal uncertainty, with human-mobility **presence** as a cross-plane leading covariate. A control layer closes the loop — a churn-capped actuator, predictive horizontal scale-out, and a true receding-horizon Model Predictive Controller (MPC) — and stays **advisory** in deployment. Across reproducible studies the system cuts worst-case overload substantially: min-max placement reduces peak UpfOI $\sim 24\%$; a learned resource-aware objective cuts **true** peak utilization $\sim 45\%$; calendar-aware forecasting reaches $+98.8\%$ skill over a random-walk baseline; capacity pooling across anti-correlated metro zones drops the worst UpfOI from 70.0 (High Stress) to 41.3 (Low Stress) with no new hardware; and MPC holds peak overload where a greedy predictive rule blows past it. We detail the models, the pedagogy behind each choice, the implementation, and the studies, with extended technical material in appendices.

Contents

1 Introduction and motivation	2
2 The optimization problem: UPF load balancing	3
2.1 The overload indicator and the goal	3
2.2 Mathematical model	3
2.3 Algorithm and solver stack	4
2.4 The resource-aware objective	4
3 The forecasting engine	5
3.1 The demand model — a multi-scale, learnable environment	5
3.2 Exogenous covariates and factor recovery	5
3.3 Mobility — the upstream, cross-plane leading driver	5
3.4 Ensemble and uncertainty quantification	6
4 The controller / advisor	6
4.1 Stance and timescales	6
4.2 Closed loop and predictive scale-out	6
4.3 True MPC — receding horizon	6
5 Studies and results	7
5.1 Optimization: placement, resource-awareness, and scale	7
5.2 Forecasting: skill, calibration, and mobility	9
5.3 Control: predictive scale-out and MPC	10

6 Conclusion and future work	11
A Appendix: PDU sessions, QoS flows, and the “flow” abstraction	13
B Appendix: Within-class heterogeneity and the realization layer	13
C Appendix: Network sizing and compute scale	13
D Appendix: Clustered / locality-constrained formulation	14
E Appendix: The forecasting method zoo	14
F Appendix: Resource-aware objective — formal statement	14
G Appendix: Receding-horizon MPC — full formulation	15
H Appendix: The metro coupled-twin scenario	15
I Supplementary material: deployment interfaces and parameters	16
Acknowledgements	18
References	18

1 Introduction and motivation

The 5G core separates a **control plane** (signalling) from a **user plane** (data). Three network functions frame this work (Table 1): the **AMF** (access and mobility management) terminates UE signalling; the **SMF** (session management) owns the PDU-session lifecycle and **selects the UPF(s)**; and the **UPF** (user-plane function) is the data-plane workhorse — **every user packet traverses it**. Once a session is established, the AMF and SMF step out of the data path entirely; all subsequent packets flow only through the UPF.

NF	Role	Load scales with
AMF	UE contact point: registration, connection state, paging, handover. Per-UE, not per-session.	Event rate — registrations/s, handovers, paging
SMF	Owns the PDU-session lifecycle; allocates IP, selects the UPF(s) , programs them over N4/PCF.	Session-setup rate + N4/PCF churn
UPF	Data-plane workhorse; anchors the session between RAN (N3) and data network (N6). Makes no policy decisions.	Throughput (pkts/s, bytes/s) + active session count

Table 1: The three network functions. The UPF is the optimization target; its load has two axes — session count and throughput.

A UPF is a **long-lived instance**, not a per-session entity: a single instance concurrently anchors tens of thousands of PDU sessions, each just kilobytes of forwarding state. The deployment unit is therefore the **instance**, provisioned with a fixed CPU/memory/NIC envelope that many sessions share; “horizontal scale-out” means **add another UPF instance**. UPFs are deployed in **regional clusters at the edge** — circles (state-sized) and, inside a metro, zones (business vs. residential) — so placement is naturally cluster-local.

The problem. The SMF’s native UPF selection is deliberately local: it picks a UPF per session from static-ish inputs (DNN / slice, tracking area, configured weights) at the instant a session arrives, with no global, forward-looking view of aggregate load. A hot slice or a skewed selection hash drives “elephant” flows onto one anchor, and **one UPF overloads while another idles**.

The approach. We compute, each decision epoch, a **placement** of the aggregate session/traffic pool across the available instances that **minimizes the worst-case UPF overload**, and we do so on a **forecast** of the coming epoch so capacity is positioned **before** a surge lands. This is **predict-then-optimize**: a forecasting engine predicts demand, a convex optimizer rebalances and scales against that prediction, and a control layer turns the plan into bounded action while staying advisory to the operator. Figure 1 shows the loop.

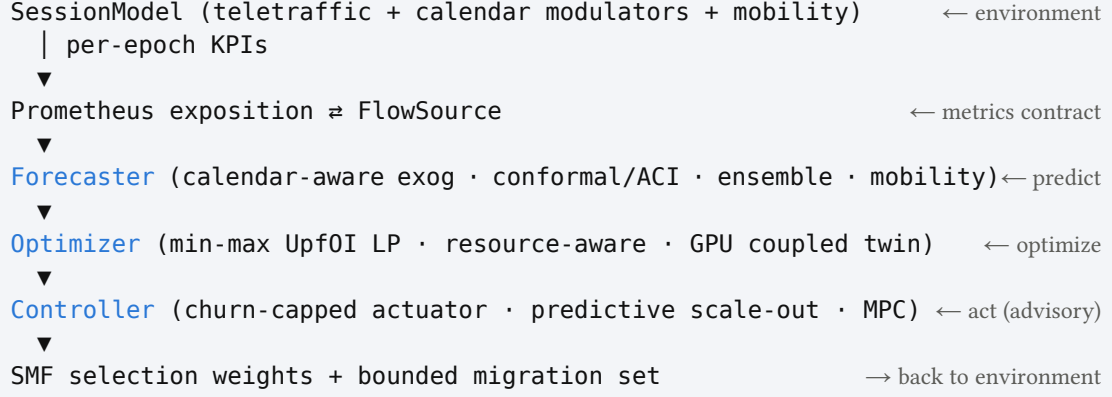


Figure 1: The predict-then-optimize closed loop. In deployment the controller emits an **advisory** (weights + a bounded migration set) to the EMS/operator; the loop is closed autonomously only in simulation, for falsifiable A/B evaluation.

The remainder of the paper is organized as the three engines and their evidence. Section 2 develops the **optimization problem** (the min-max LP, its solver stack, and the learned resource-aware objective). Section 3 develops the **forecasting engine** (the demand model, exogenous covariates, mobility, and uncertainty). Section 4 develops the **controller** (advisory stance, closed loop, predictive scale-out, and MPC). Section 5 consolidates the **studies and results**. Section 6 concludes. Appendices A–H carry the detailed technical expositions — the “flow” abstraction and realization layer (A, B), network sizing (C), the clustered formulation (D), the forecasting zoo (E), the resource-aware and MPC formalisms (F, G), and the coupled-twin scenario (H) — and a supplementary section (Section I) documents the deployment interfaces and parameters.

2 The optimization problem: UPF load balancing

2.1 The overload indicator and the goal

The reference design scores each UPF with a scalar health proxy, the **UPF Overload Indicator**:

$$\text{UpfOI}_j = 100 \cdot (0.4 \cdot \text{normSessionLoad}_j + 0.6 \cdot \text{normThroughputLoad}_j), \quad (1)$$

with verdict bands < 50 Low Stress, 50–60 Moderate, > 60 High Stress. The two terms are **workload proxies**: session load (active sessions / capacity) proxies memory / control-state pressure; throughput load (packet rate / capacity) proxies CPU / NIC pressure. The 0.4/0.6 blend encodes “throughput stresses our build more than session count.” Optimizing over **workload** rather than reading `nf_cpu_mc` / `nf_memory_mb` directly is deliberate: workload **leads** resources and forecasts better, and the optimizer’s lever **is** moving sessions/traffic. (The fixed blend is only a proxy; Section 2.4 **learns** the right per-UPF weighting.)

The goal, each epoch: compute a placement of the aggregate pool across the available UPFs that **minimizes the worst-case UpfOI**, so that after redistribution every UPF sits in a healthy band.

2.2 Mathematical model

We place **session-classes**, not individual sessions. “Flow i ” denotes a **session-class** — a bucket of many PDU sessions grouped by routing-relevant attributes (DNN / slice / 5QI / locality) that the SMF steers together — so its data is a count s_i (sessions, $s_i \geq 1$) and a sum t_i (aggregate throughput). The atomic unit of **realization** is the whole PDU session (Appendix A). Per epoch the data is s_i, t_i (measured over the previous epoch, or **forecast** for the coming one) and per-UPF normalizers $S_j^{\max}, T_j^{\max}$.

The decision variable is $x_{ij} \in [0, 1]$, the fraction of class i ’s load placed on UPF j . Per-UPF loads are **linear** in x :

$$\text{SessLoad}_j = \frac{\sum_i x_{ij} s_i}{S_j^{\max}}, \quad \text{ThrLoad}_j = \frac{\sum_i x_{ij} t_i}{T_j^{\max}}, \quad \text{UpfOI}_j(x) = 100(0.4\text{SessLoad}_j + 0.6\text{ThrLoad}_j). \quad (2)$$

The objective is peak-shaving — minimize the worst UPF’s indicator, $\min_x \max_j \text{UpfOI}_j(x)$ — subject to every class being fully placed, $\sum_j x_{ij} = 1 \forall i$, $x_{ij} \geq 0$. Introducing a scalar τ removes the non-smooth max (the **epigraph** reformulation):

$$\min_{x, \tau} \tau \quad \text{s.t.} \quad \text{UpfOI}_j(x) \leq \tau \quad \forall j, \quad \sum_j x_{ij} = 1, \quad x_{ij} \geq 0. \quad (3)$$

Because UpfOI_j is linear in x and all constraints are linear, Equation 3 is a **linear program** — specifically a min-max LP. Optional refinements keep it an LP: **eligibility** masks ($x_{ij} = 0$ unless UPF j may serve class i) block-structure the problem by region; an **anchor-affinity** / **churn cap** ($\|x - x^0\|_1 \leq \text{cap} \cdot N$, a per-class L_1 migration budget relative to a prior placement x^0) bounds live-session migration; a **tier preference** ($+\lambda \sum_{ij} c_{ij} x_{ij}$, a soft “stay local” cost) breaks ties among equal-peak plans; a hard **stress cap** ($\text{UpfOI}_j \leq 60$) can be added as feasibility. These map to the `eligibility`, `x0/churn_cap`, and `pref_cost/pref_weight` arguments of the solver. The clusters give the LP two regimes — **independent** (solve each cluster separately) and **coupled** (one block-structured LP whose spillover lets clusters pool capacity; Appendix D).

2.3 Algorithm and solver stack

The model class is a continuous convex LP, solved by a primal-dual **interior-point** (Mehrotra predictor-corrector) method. Each Newton step solves a sparse KKT system; the constraint matrix is highly structured (block-diagonal conservation blocks plus M coupling rows), which interior-point methods exploit efficiently. A future quadratic fairness term would make it a QP — still convex, same solver family. The implementation layers open-source tools behind **CVXPY** as the modeling layer:

Layer	Tool	Why
Modeling	CVXPY	Declarative epigraph LP; swap solvers without rewriting
CPU solve (PoC)	HiGHS	Best-in-class open-source LP; instant for small clusters
GPU cone / IPM	CuClarabel	GPU port of Clarabel — drop-in when scaling
GPU LP at scale	cuOpt (barrier / PDLDP)	HPC path for 100+ UPFs, millions of variables
Sparse KKT engine	cuDSS	Under cuOpt / CuClarabel

Table 2: The solver stack. CVXPY as the modeling layer means the CPU model and the GPU model are **the same model** — moving to GPU is a one-word `solver=` change (for cuOpt, one wrapper call with the identical signature).

The key property is **solver-agnosticism**: the CPU path (`solver="HIGHS"`) and the GPU path differ only by the backend argument. In the PoC, HiGHS and Clarabel agree to three decimals; at the coupled-twin scale cuOpt and HiGHS agree on the optimum while cuOpt solves $\sim 8\times$ faster (Section 5). That is the bridge from a laptop to an HPC digital twin.

2.4 The resource-aware objective

The fixed 0.4/0.6 blend is a proxy that holds only while the workload→resource map is roughly linear and homogeneous. It breaks along independent axes: **across UPFs** (a DPU-offloaded UPF spends far less CPU per Mbps than a software one), **across classes** (a URLLC session can cost several times an idle eMBB one), and in **nonlinear regimes** (CPU rising super-linearly at high packet rates). The first two are repaired **without leaving the LP class**: replace the hand-tuned blend with a **learned** objective — a per-UPF (and optionally per-slice) regression $\widehat{\text{CPU}}, \widehat{\text{MEM}} = f(\text{sessions}, \text{throughput})$ fitted from `nf_cpu_mc / nf_memory_mb` — and minimize the **worst true utilization**:

$$\min_{x, \tau} \tau \quad \text{s.t.} \quad \widehat{\text{CPU}}_j \frac{x}{C_j^{\text{cpu}}} \leq \tau, \quad \widehat{\text{MEM}}_j \frac{x}{C_j^{\text{mem}}} \leq \tau \quad \forall j. \quad (4)$$

Sessions and throughput are linear in x , so CPU and memory are too — the LP simply bounds **both** resources by τ instead of one blended scalar (now $2M$ epigraph rows). The coefficients are fit **per UPF** by non-negative least squares from the observed gauges — one regression per UPF, so hardware heterogeneity is captured — and the fit’s **implied** UpfOI weights read back the true blend (on the demonstrated fleet, throughput-weight ~ 0.88 for software vs. ~ 0.72 for DPU-offloaded UPFs, vs. the fixed 0.60). A **per-slice** variant indexes the coefficients by 5QI, **ridge-shrunk** toward the per-UPF estimate so thin or collinear data collapses gracefully rather than being exploited. This is a slow **calibration** (refit on an hours-to-days cadence, persisted and reloaded), not a per-epoch step, and it runs at the collector/HPC layer, never on the SMF hot path. Full statement — the fit, the implied weights, the per-slice model — in Section F; measured results in Section 5.

3 The forecasting engine

The optimizer runs best on the **coming** epoch’s demand, so it is fed a forecast. The engine is a **general multivariate time-series forecaster** — interchangeable methods behind one `fit(X) → forecast(steps)` interface, plus composable wrappers for exogenous covariates, uncertainty, and ensembling — operating on any (T, D) signal. UPF demand (s_i, t_i) is the worked example.

3.1 The demand model — a multi-scale, learnable environment

Forecasting is only interesting if the signal has learnable structure. For each class c the session count is an $M/M/\infty$ **birth-death process**: arrivals are a non-homogeneous Poisson process of rate $\lambda_c(t)$ (the time-varying parameter carrying all calendar/mobility structure), and each active session departs at rate $\mu_c = 1/H_c$ (a **fixed** mean holding time H_c). The stationary occupancy is Poisson, so by Little’s law

$$s_c(t) = N_c(t) \sim \text{Poisson}(m_c(t)), \quad m_c(t) = \lambda_c(t) H_c. \quad (5)$$

Throughput is occupancy times a **fixed** per-class rate r_c with mild jitter, $t_c(t) = N_c(t) \cdot r_c \cdot \xi_c(t)$ with $\xi_c \sim \text{LogNormal}(0, \sigma^2)$, so **throughput inherits its temporal structure from occupancy**. Structure enters **multiplicatively on the rate** — a product of mean-1 modulators, i.e. **additive in log**:

$$\lambda_c(t) = \lambda_c^0 \prod_k g_k(t)^{\beta_k} \rightarrow \log \lambda_c(t) = \log \lambda_c^0 + \sum_k \beta_k \log g_k(t), \quad (6)$$

with β_k the **elasticity** of the rate w.r.t. factor k . Each factor being mean-1 over its own period makes the decomposition **identifiable**: the base λ_c^0 carries the level, each g_k only its shape. This log-additivity is exactly why a log-linear regression recovers each factor as e^{β} . The factors split into **endogenous** (recurring, learnable from history: diurnal / weekly / yearly seasonals, short-range autocorrelation) and **exogenous** (external, entering as known-future covariates: holidays, events, mobility, weather).

3.2 Exogenous covariates and factor recovery

Periodic structure **can** be learned endogenously, but holidays and events **cannot** — they are irregular calendar interventions, predictable only as **known-future covariates**. The pattern is therefore **regress-then-model-residual**: an explicit ordinary-least-squares regression in log-space on a design matrix of known-future features. With $Y = \log X$ and feature matrix F :

$$Y = FB + R, \quad \hat{B} = \arg \min_B \|Y - FB\|^2, \quad \hat{X} = \exp(F_{\text{future}} \hat{B} + \hat{R}). \quad (7)$$

F (the design matrix) has one column group per factor: a small Fourier basis (3/3/4 harmonics for diurnal/weekly/yearly, both cos and sin), a 0/1 holiday flag, a 0/1 event flag, an intercept, and optionally a linear trend — 23 columns by default, all computable for **future** times. B (the fitted coefficients) doubles as **factor recovery**: because the fit is in log-space, e^{β} are the multipliers, so the same model that forecasts also reads back the generative factors. R (the residual) — what the calendar cannot explain (the AR(1) log-noise plus any un-modeled structure) — is handed to **any** base method (AR, Koopman, neural, ...). The wrapper is itself a Forecaster, so exogenous support reaches the whole method zoo. Adding a factor (weather, promotions, price) is a one-line feature addition, never a modeling rewrite.

3.3 Mobility — the upstream, cross-plane leading driver

Mobility is the upstream driver of demand’s spatial-temporal structure: people move → per-zone **presence** shifts → arrivals shift → load shifts. It enters through occupancy as a mean-1 modulator on $\lambda_c(t)$. A zone z has home population home_z and work population work_z , and a fraction $a(t)$ is “at work” on a commute schedule; presence is the convex mix

$$\text{presence}_z(t) = (1 - a(t)) \text{home}_z + a(t) \text{work}_z, \quad \text{work}_z = (1 - \varphi) \text{home}_z + \varphi \sum_{z'} \text{home}_{z'} P_{z' \rightarrow z}, \quad (8)$$

with $P_{z' \rightarrow z}$ the origin–destination kernel from a parameter-free **radiation** model. A business zone (many jobs, few residents) fills at midday; a residential zone at night — so the **anti-phase emerges** from the OD coupling, presence conserved. A MobilityEstimator learns the inter-zone coupling and forecasts presence, which then feeds the UPF forecaster as a **known-future covariate**. Because presence is **AMF-observable** and **leads** load (arrivals precede sessions), it is a **cross-plane leading signal**: we predict the commute from AMF

presence rather than re-learn it from noisy UPF demand. This is what makes cross-zone capacity pooling pay off downstream (Section 5).

3.4 Ensemble and uncertainty quantification

No single base method wins across regimes — the ranking flips between clean and bursty traffic (Section 5) — so the engine **ensembles**: members are scored out-of-sample on a held-out validation tail and blended by inverse-MAE, $\hat{y}_{\text{ens}} = \sum_m w_m \hat{y}_m$ with $w_m \propto 1/(\text{MAE}_m + \varepsilon)$, so a member that collapses in the current regime is automatically down-weighted. The weighted **spread** of members is a free model-disagreement band.

A robust optimizer provisions for a **worst plausible** load, so the forecaster must emit a **calibrated band**, and “calibrated” must be measured. **Split-conformal** records, per horizon h , the empirical distribution of absolute residuals on a rolling-origin pass over a held-out tail; the interval is $\hat{y}_h \pm \hat{q}_h$ with $\hat{q}_h = \text{Quantile}_{1-\alpha}$ of that horizon’s errors. Under calendar-driven shift a fixed quantile mis-calibrates (measured 99% and 82% at a 90% target). **Adaptive Conformal Inference** (ACI) restores nominal coverage with a rolling score window and an online level adaptation,

$$\alpha_{t+1} = \alpha_t + \gamma((1 - \text{target}) - \mathbb{1}[y_t \notin \text{interval}_t]), \quad (9)$$

driving long-run coverage to nominal under shift. The base-method “zoo” (persistence, seasonal-naive, AR, Holt-Winters, Koopman/DMD, ARIMA, neural MLP) is detailed in Appendix E; all satisfy one interface, so exogenous support, ensembling, and conformal intervals compose over any of them.

4 The controller / advisor

4.1 Stance and timescales

The controller turns the optimizer’s output into **action**. In deployment it is **advisory**: the optimizer runs out-of-band on the HPC, consuming aggregated/predicted KPIs and emitting a **recommendation** — SMF selection weights plus a bounded migration set — to the EMS or a human. It never sits on the SMF’s per-session hot path. Only in **simulation** is the loop closed autonomously, for falsifiable policy A/B comparison. This is **not** reinforcement learning; it is predict-then-optimize plus a per-epoch convex re-solve, across three separated timescales: per-session steering (ms, SMF), per-epoch re-solve (~ 1 min, HPC optimizer), and per-scale-event provisioning (rare, EMS). Steering **new** sessions is cheap; migrating **live** sessions is disruptive — hence a **churn cap** bounding how many existing sessions a rebalance may move, trading optimality for stability.

4.2 Closed loop and predictive scale-out

The closed loop is `environment.step() → policy → SMF actuator.apply → KPIs`. The environment is open-loop w.r.t. placement (arrivals don’t depend on where sessions anchor), so two seeds see the same trajectory — a fair A/B. The actuator freely steers new sessions but caps live migration at `churn_frac` per epoch. Where prediction is decisive is **when to add an instance**: a new UPF has spin-up **latency** L , so a fast surge overloads for $\sim L$ epochs unless capacity is provisioned ahead. Three policies — **static** (fixed M ; overloads through surges), **reactive** (scale on breach; overloads for $\sim L$ epochs), and **predictive** (forecast $H \geq L$ ahead and scale before the surge) — bracket the trade-off.

4.3 True MPC — receding horizon

The greedy predictive rule sizes M from a **single** forecast point H ahead, which (i) under-provisions a surge peaking **between** now and H , and (ii) **flaps** M against a hard target on demand that wiggles. True MPC closes both: each epoch it optimizes the whole **availability trajectory** $m_0, \dots, m_H$ over the H -epoch horizon (m_k = instances available at step k ; $m_0 = M_{\text{now}}$ fixed), applies only the **first** action, and re-plans:

$$\min_{m_1, \dots, m_H} w_{\text{ov}} \sum_{k=0}^H (\ell_k/m_k - \text{target})_+ + \lambda_i \sum_{k=1}^H m_k + \lambda_c \sum_{k=1}^H |m_k - m_{k-1}|, \quad (10)$$

where ℓ_k is the **forecast load** (the balanced-UpfOI numerator) so ℓ_k/m_k is the balanced UpfOI at availability m_k , and **target** is the UpfOI **operating point** overload is measured against (default 50, below the 60 High-Stress threshold — **not** the 0.6 throughput weight). The three terms trade **overload** against **capacity** (λ_i minimizes total provisioned instance-epochs) and **switching** (λ_c , an L_1 total-variation anti-oscillation cost).

Spin-up latency enters the dynamics (m_k can exceed committed capacity only for $k \geq L$); scale-in is immediate, one instance/epoch. Each term is convex (overload via `inv_pos`, the convex reciprocal $1/m$), so every step is a small convex program (CLARABEL), integer M by rounding. There is **no terminal cost** — a receding re-plan, $H \geq L$, and a reactive safety floor (never below the current need) keep it well-behaved without a formal stability claim. Greedy predictive is exactly the $\lambda_c = 0$, single-action ($H = L$) special case — MPC strictly generalizes it, and the switching term buys the smoothness. Full statement in Section G; results in Section 5.

Every policy is scored on the **same fixed KPI set** — peak UpfOI, time-above-threshold, churn, oscillation index, forecast skill, interval quality, SLA proxies — so “better than baseline” is falsifiable. The stability/optimalty trade-off is set by separate **knobs** (churn cap, hysteresis, horizon H); the KPIs tell you whether the knobs are right.

5 Studies and results

Each study is a runnable script (a console command plus an importable `run_*`). We group them by engine. All figures regenerate from live runs.

5.1 Optimization: placement, resource-awareness, and scale

Min-max LP PoC ($M = 2$ UPFs, $N = 300$ heavy-tailed classes). A skewed baseline leaves the worst UPF at **72.55** (High Stress) while its peer idles at 37.45; the min-max LP cuts the worst to **55.18** (both Moderate) — a $\sim 24\%$ peak reduction produced by an optimizer, not a hand-tuned move. The atomic-realization rounding gap is $+0.18$ (the continuous relaxation is tight), and HiGHS $\tau =$ Clarabel $\tau = 55.000$ confirms solver-agnosticism.

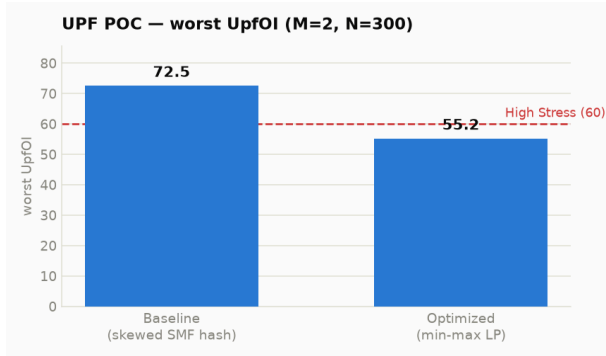


Figure 2: Min-max LP PoC: worst UpfOI, skewed baseline vs. optimized.

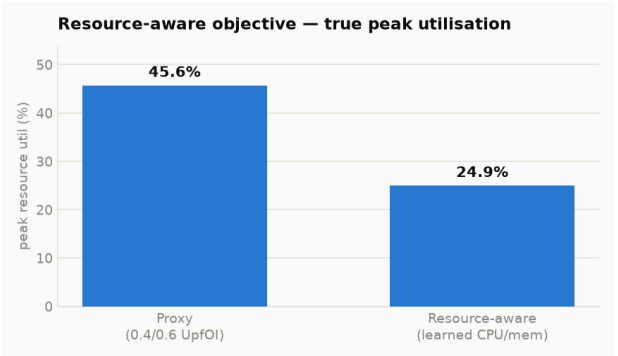


Figure 3: Resource-aware objective: **true** peak utilization, workload proxy vs. learned CPU/mem.

Resource-aware objective (heterogeneous fleet; software UPFs burn $\sim 3\times$ the CPU/Mbps of DPU-offloaded ones). The fit recovers the true coefficients to $\sim 3\%$ under gauge noise; the implied UpfOI throughput-weight comes out **0.71–0.89 and varies per UPF** (vs. the fixed 0.60), so no single 0.4/0.6 is right. Judged by **true** resources, the learned objective cuts peak utilization **45.6% $\rightarrow$ 24.9%** ($\sim 45\%$) — it steers throughput onto the cheap DPU-offloaded UPFs, which the workload proxy cannot see. A **per-slice** variant separates eMBB (0.46 mc/session) from URLLC (6.05) from **aggregate** CPU alone, cutting true peak utilization $49.9\% \rightarrow 43.1\%$, and **degrades gracefully** under collinear data (the recovered spread collapses $5.6 \rightarrow 1.4$) rather than fitting noise (Figure 4; formalism in Section F).

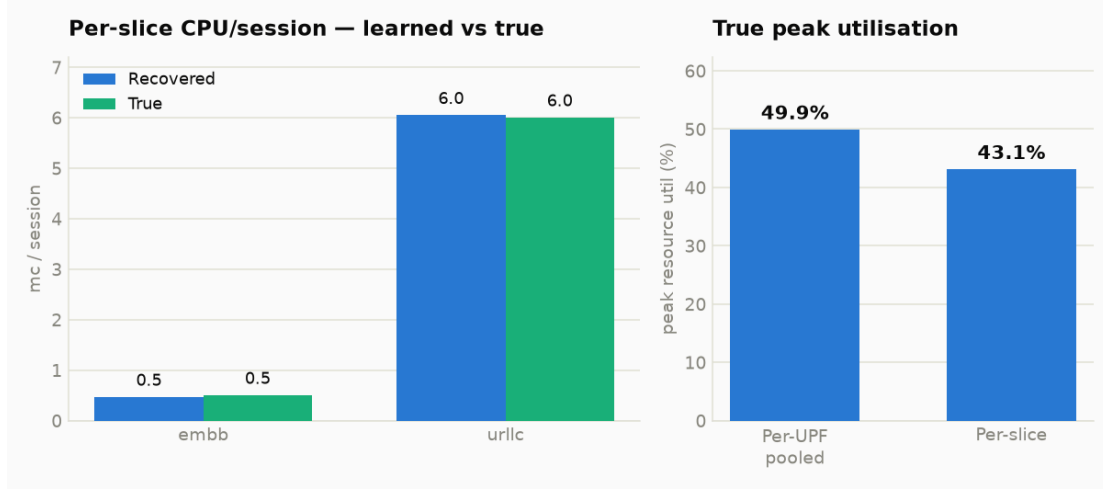


Figure 4: Per-slice resource objective: from **aggregate** per-UPF CPU the fit separates the CPU-cheap eMBB slice from the costly URLLC slice, and placing the costly sessions knowingly cuts true peak utilization 49.9% → 43.1%. Under collinear (non-identifiable) data the per-slice spread shrinks back toward the pooled per-UPF estimate — graceful degradation, not noise-fitting.

Operator-scale coupled twin. This is the study that speaks to a real deployment. A **metro** twin places **8,000** class-loads across **48 UPFs** (8 zones × 6) as **one coupled 384,001-variable min-max LP** every epoch, learned over 3 years at hourly cadence. The learned inter-zone load-correlation matrix has a clean block structure — **within-archetype +0.99, cross-archetype (business↔residential) -0.98** — the commuter signature. At the business-district peak, solving each zone **independently** leaves the worst UPF at **70.0 (High Stress)** while residential UPFs idle; the **coupled twin** pools that idle capacity and cuts the worst UpfOI to **41.3 (High → Low Stress)** — with no new hardware, purely by solving the metro as one problem. This is the headline operator result. The scenario's structure — the commute OD matrix and the **block-diagonal (independent) vs. tridiagonal (coupled)** eligibility A — is Figure 5; the full construction (nodes, radiation OD, eligibility, fixed 6 UPFs/zone) is Section H.

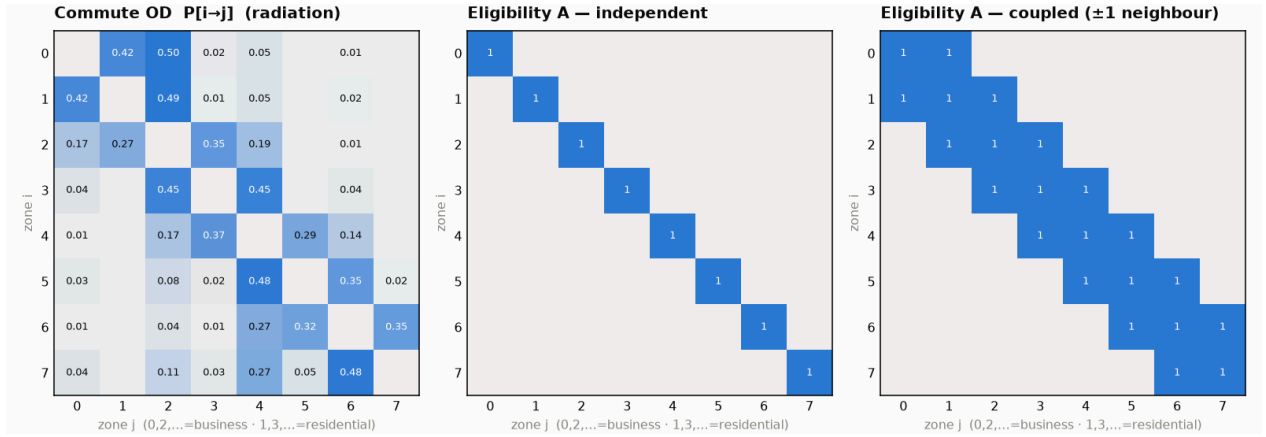


Figure 5: The coupled-twin scenario, 8 metro zones on a line (even = business, odd = residential). **Left:** the radiation commute OD P_{ij} (residential commuters flow to job-rich business zones). **Centre / right:** the eligibility mask A — **independent** (block-diagonal: own-zone UPFs only) vs. **coupled** (tridiagonal: own zone plus the two neighbours). Because zones alternate archetype, every spillover edge joins a business zone to an anti-phased residential one — the structural reason pooling pays.

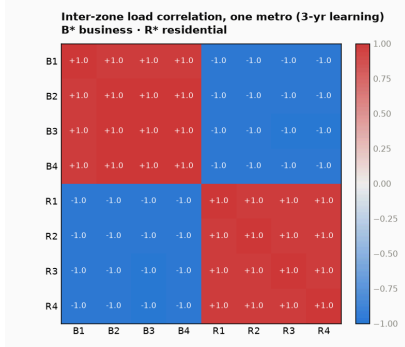


Figure 6: Inter-zone load correlation: +0.99 within, -0.98 across archetypes.

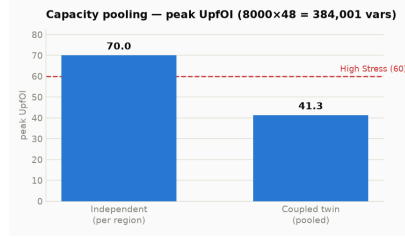


Figure 7: Independent vs. coupled worst UpfOI: 70.0 $\rightarrow$ 41.3.

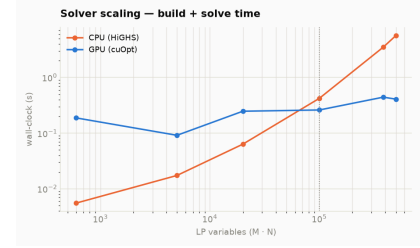


Figure 8: CPU vs. GPU solver scaling; crossover near 10^5 variables.

Does the GPU pay off? Sweeping problem size confirms a crossover near 10^5 variables: HiGHS is faster for small problems but its build+solve grows steeply, while cuOpt stays roughly flat. At the 384k-variable metro twin cuOpt solves in **0.44 s vs. HiGHS 3.5 s** ($\sim 8\times$) and agrees on the optimum; by 2M variables the gap is $\sim 50\times$ wall-clock. So the coupled twin is genuinely GPU-worthy, and the one-line solver swap (Section 2.3) is what makes the national tier (millions of variables) deployable on the HPC.

5.2 Forecasting: skill, calibration, and mobility

Method comparison — regime-dependent. On **clean** diurnal traffic, seasonal-naive wins (+97.9% skill over persistence) but under **bursts** it is the worst (-70.8%), while AR is the most robust (+43.8%). No method dominates — the motivation for the ensemble. **Calendar-aware vs. endogenous:** wrapping **any** base with the exogenous calendar dominates — **exog_ar** reaches **+98.8%** skill and even **exog_persistence** (+98.0%) beats the best endogenous learner — because once the calendar is removed the residual is nearly white. **Factor recovery** validates the model: recovered holiday depressor 0.551 (true 0.55), event excitor 2.177 (true 2.2), weekly profile to within 0.004. **Intervals:** split-conformal mis-calibrates under shift (99%/82% at a 90% target); **ACI restores nominal 90.0% coverage**, and a better base is $\sim 11\times$ sharper at the same correct coverage.

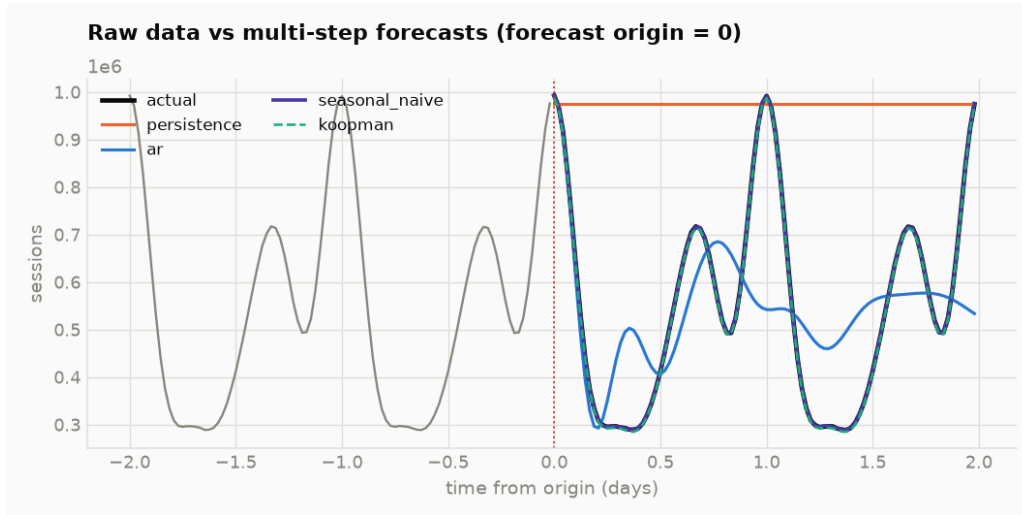


Figure 9: Base-method comparison across regimes: which method wins **flips** between clean and bursty traffic — seasonal-naive is best on clean seasonality and worst under bursts, AR is the most robust. No method dominates, which is exactly what motivates the ensemble.

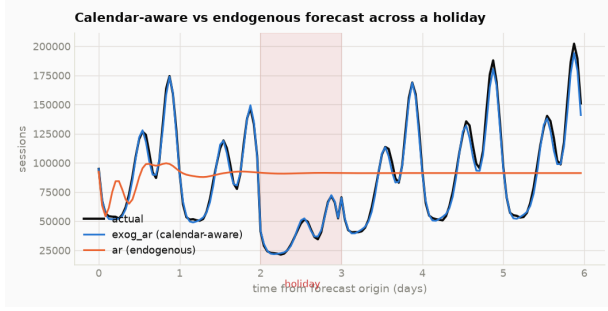


Figure 10: Calendar-aware (exog_ar) vs. endogenous (ar) across a holiday — the exog wins where the calendar bites.

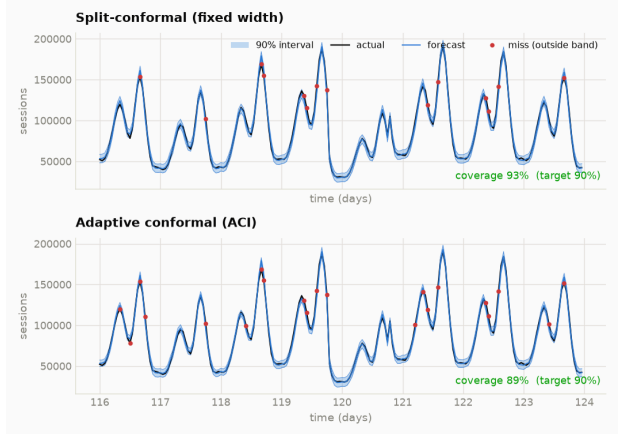


Figure 11: Split-conformal (fixed width) vs. ACI (adaptive) 90% intervals; “misses” marked.

Mobility-aware forecasting. Across an **unscheduled crowd surge** the mobility-aware forecaster (calendar + presence covariate) tracks the spike while the calendar-only forecaster misses it entirely — presence is known-future and leads the load. Mobility-awareness improves UPF-demand MAE $\sim 14\%$, and the learned presence elasticity (0.99) validates $t_c \propto \text{presence}$.

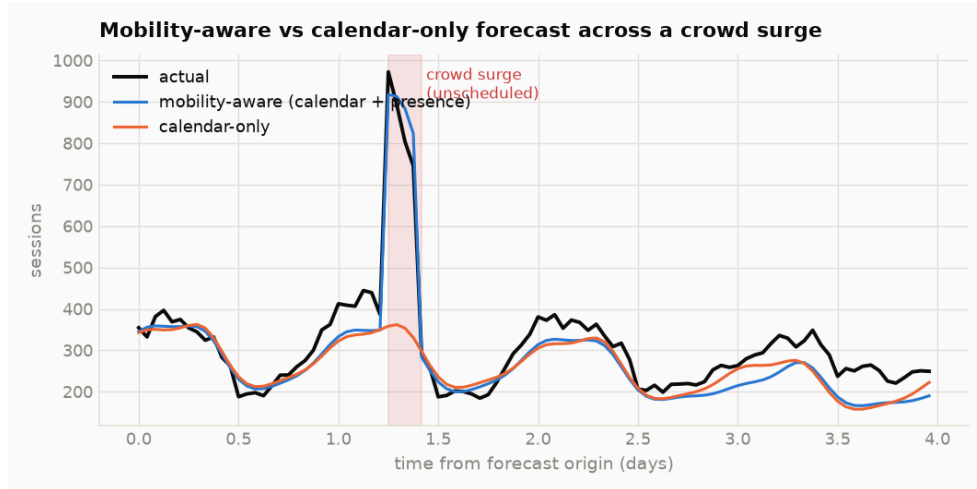


Figure 12: Mobility-aware vs. calendar-only forecast across an unscheduled crowd surge. Presence (AMF-observable, leading) lets the forecaster anticipate a burst the calendar never scheduled.

5.3 Control: predictive scale-out and MPC

Predictive scale-out cuts overload time **static 8.4% $\rightarrow$ reactive 4.5% $\rightarrow$ predictive 1.4%** ($\sim 3\times$ below reactive) at the honest cost of $\sim 11\%$ more instance-epochs; turning on the intra-UPF min-max LP rebalancing under predictive keeps overload at **1.5%** — the two levers compose (below, left). Over time (right), static rides every surge past High Stress, reactive overloads for the $\sim L$ spin-up window at each onset, and predictive stays under the line except during the first ~ 2 -day forecaster warm-up.

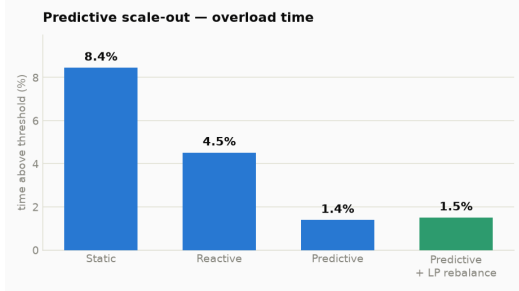


Figure 13: Overload time by policy; the 4th bar adds real LP rebalancing under predictive (1.4% $\rightarrow$ 1.5%).

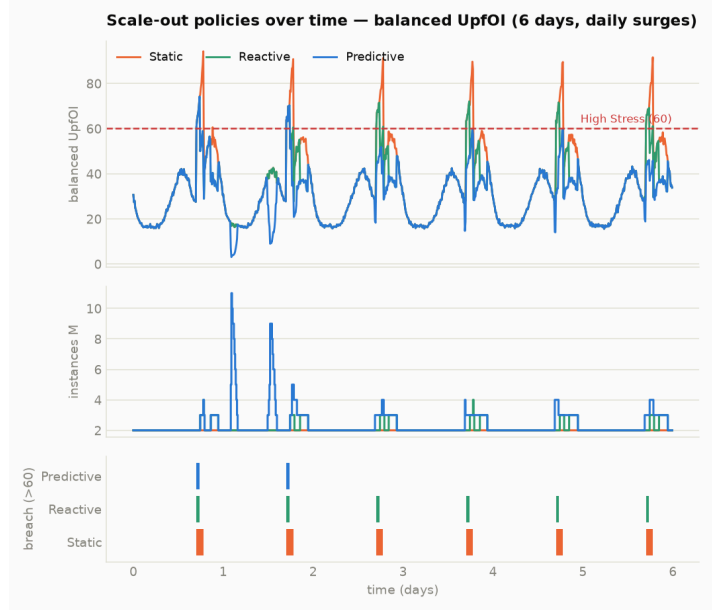


Figure 14: The three policies over time: balanced UpfOI, instance count M , and a per-policy breach strip.

True MPC helps most where the greedy rule is weakest. On **closely-spaced surges** (gaps shorter than the horizon, where the single-point rule misses mid-horizon peaks and flaps), MPC cuts peak UpfOI **120.2 $\rightarrow$ 88.3**, time-above-threshold **3.94% $\rightarrow$ 1.27%**, and M -oscillation **16 $\rightarrow$ 1 reversals**, robust across seeds; on well-separated surges the edge is modest ($\sim 7\%$ less capacity, $\sim \frac{1}{2}$ the churn at equal peak). The overload/churn trade-off is now a tunable cost rather than a fixed cap.

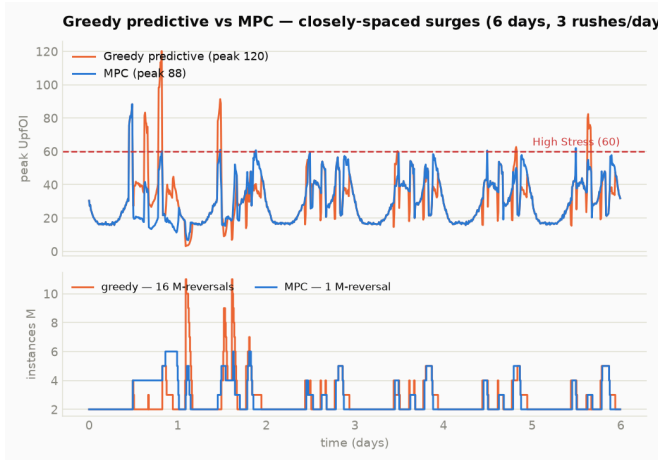


Figure 15: Greedy predictive vs. MPC over time on closely-spaced surges: MPC holds the peak (top) and stops the M hunting (bottom).

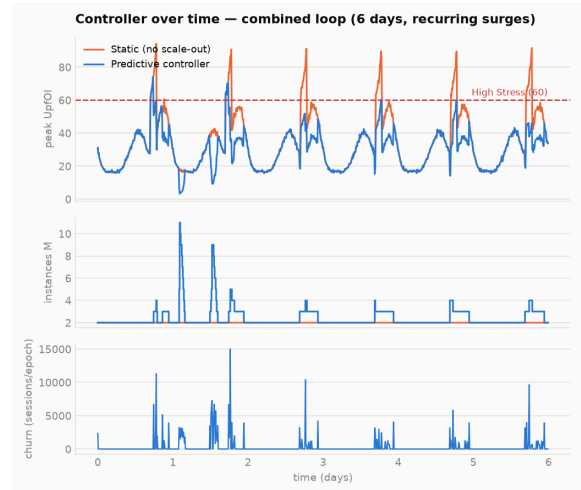


Figure 16: Combined loop: static (fixed M) vs. the predictive controller — peak UpfOI, instances, and churn over 6 days.

Results at a glance

Min-max placement: peak UpfOI -24% · Resource-aware: true peak utilization -45% · Calendar-aware forecasting: $+98.8\%$ skill · ACI: 90.0% coverage restored, $\sim 11\times$ sharper · Metro pooling: worst UpfOI 70.0 $\rightarrow$ 41.3 (High $\rightarrow$ Low) · GPU: $\sim 8\times$ at 384k vars · Predictive scale-out: overload time 8.4% $\rightarrow$ 1.4% · MPC: peak 120 $\rightarrow$ 88, oscillation 16 $\rightarrow$ 1.

6 Conclusion and future work

We have built a **predict-then-optimize** control system for the 5G user plane that positions capacity **ahead** of overload. Its spine is a **convex min-max LP** that stays solver-agnostic from a laptop to a GPU HPC; a **composable forecasting engine** that is calendar-aware, mobility-aware, ensembled, and calibrated; and an **advisory control layer** — churn-capped actuation, predictive scale-out, and true MPC. The evidence

is consistent: worst-case overload falls across placement, resourcing, forecasting, pooling, and control, each demonstrated by a reproducible study.

Three design commitments recur. **Convexity is a feature**: every extension (eligibility, churn cap, resource-awareness, per-slice costs, MPC) was engineered to stay in the LP/QP class, buying GPU scalability and predictable solve times. **Measurement precedes optimization**: a fixed KPI set makes every claim falsifiable. **Advisory by design**: the deployed system recommends; a human or EMS disposes.

Future work. (i) **On-cluster HPC deployment** against a live production Prometheus — a URL/auth swap, since the metrics contract is already spoken — and the **national tier** (~ 22 circles, ~ 5.8M variables). (ii) **Wiring mobility presence into the live loop** (a feature-composition change once an AMF presence feed exists). (iii) **The control-plane sibling** (AMF/SMF): the same forecasting and optimization machinery transfers to registration / handover / paging, where mobility becomes a first-class trajectory-crossing driver. (iv) **Nonlinear resource regimes** via a piecewise-linear or convex objective extension.

Reproducibility. All studies are console scripts (run-upf-poc, resource-study, forecast-compare, forecast-region-study, mobility-study, scaling-demo, mpc-study, stress-test, bench-scaling); figures regenerate via `python docs/figures/make_figures.py`. Source: github.com/amrutur/telecom_optimizers. **Cite as:** B. Amrutur, “telecom_optimizers: predict-then-optimize 5G traffic engineering (UPF load balancing),” Indian Institute of Science, Bangalore, 2026. CC BY-NC 4.0.

A Appendix: PDU sessions, QoS flows, and the “flow” abstraction

The 3GPP hierarchy is $\text{UE} \rightarrow \text{PDU session} \rightarrow \text{QoS flow (QFI)} \rightarrow \text{SDF}$. A **PDU session** is a UE’s connection to one data network, with one IP and anchored at exactly one **PSA-UPF**; inside it are one or more **QoS flows** (each a QFI with its own 5QI/ARP), each carrying one or more SDFs (5-tuple application flows). **All QoS flows of a PDU session traverse the same PSA-UPF** — you cannot load-balance them across UPFs — so the atomic, indivisible unit for **placement** is the **whole PDU session**. In this paper “flow i ” is a **session-class** (a bucket of many PDU sessions grouped by DNN / slice / 5QI / locality); x_{ij} reads “what fraction of class i ’s sessions go to UPF j ,” and atomic realization rounds that back to routing whole PDU sessions. The continuous relaxation is valid precisely because each class holds many sessions (the rounding gap is $O(1)$ session). Advanced topologies (UL Classifier, Branching Point, multi-homing) insert an additional I-UPF for local breakout — a different role, not peer load-balancing — and are out of Phase-1 scope.

B Appendix: Within-class heterogeneity and the realization layer

The optimizer partitions a class across UPFs; it does not say which individual session goes where — the SMF does that, minimizing churn. Sessions differ even within a class (a 4K streamer vs. an idle sensor), handled not by per-session accounting but by **average-cost accounting plus a class taxonomy**. The inputs s_i, t_i are measured totals, so the class carries a real observed cost (mean per-session throughput $\bar{t}_i = t_i/s_i$); what is dropped is the within-class **variance**. This suffices when (i) the **law of large numbers** engages — a representative fraction x_{ij} gives realized load $\approx x_{ij}t_i$ with variance shrinking as sessions-per-class grows; and (ii) the **class taxonomy** is fine enough that within-class sessions are roughly exchangeable (split by DNN $\rightarrow$ slice $\rightarrow$ 5QI $\rightarrow$ app until homogeneous — this leaves the LP structurally unchanged, only N grows). It **breaks** under heavy tails (a few elephant sessions dominate t_i , so **which** sessions are placed matters) or thin classes (LLN doesn’t engage). A deep reason churn is unavoidable: at session establishment the SMF **does not yet know the session’s throughput**, so it places on priors and inevitably mis-guesses; the slow loop (measured aggregates $\rightarrow$ re-solve $\rightarrow$ migrate a few) corrects it, mediated by the churn cap. If within-class cost must become first-class: **finer classes** (preferred; same LP, larger N), **robust/headroom** terms (a QP/SOCP, same solver family), or **per-session MILP** (combinatorial, NP-hard, deliberately avoided).

C Appendix: Network sizing and compute scale

UPFs are **distributed to the edge**, not centralized: a national operator runs hundreds to low-thousands of instances across a hierarchy of sites (national core DC $\rightarrow$ regional/circle DC $\rightarrow$ metro/aggregation edge $\rightarrow$ far-edge/MEC). For a Jio-class operator ($\sim 490\text{M}$ subscribers, $\sim 0.6\text{--}1$ billion concurrent PDU sessions, $\sim 85\text{--}120$ Tbps peak), an order-of-magnitude instance count builds from the throughput floor with **utilization headroom** — the UpfOI Low/Moderate band — as the dominant driver: $(120 \text{ Tbps} / 100 \text{ Gbps}) \times (1/0.65) \times 1.1 \approx 2,000$ instances. Crucially, **per-decision M and N are small**: sessions steer only among UPFs valid for their locality/slice, so the national problem partitions into many independent regional problems with modest pools ($M \approx$ single digits to low tens for metro MBB), and N counts session-**classes**, not sessions. The HPC earns its keep not from one giant solve but from **breadth** (thousands of regional problems re-solved every minute), a **national joint twin** (coupling regions into a $10^7\text{--}10^9$ -variable sparse LP), and **predict-then-optimize at scale** (forecasters over millions of series). Measured (RTX PRO 6000 Blackwell vs. i9-14900KS):

variables ($M \cdot N$)	HiGHS	Clarabel	cuOpt (GPU)
5 k	0.02 s	0.02 s	0.09 s
100 k	0.42 s	1.3 s	0.26 s
500 k	5.7 s	12 s	0.49 s
2 M	74 s	94 s	1.4 s

Table 3: Wall-clock build+solve for the min-max LP. Measured CPU $\rightarrow$ GPU crossover $\sim 10^5$ variables; $\sim 50\times$ at 2M.

D Appendix: Clustered / locality-constrained formulation

Locality is a **property of the class**, not a new constraint layer: extend the class key to (DNN $\times$ slice $\times$ 5QI $\times$ service-area/TAI-group), giving each class an eligible UPF set E_i (a 0/1 mask A_{ij}). The generalized LP is Equation 3 with $\sum_{j \in E_i} x_{ij} = 1$ and $x_{ij} = 0$ off E_i — still an LP, just a sparse x . **Two regimes**: hard pinning (block-diagonal A) separates the LP **exactly** into independent per-cluster solves — Appendix C’s embarrassingly-parallel regional problems; coupling (general A) keeps one sparse LP only when clusters are genuinely entangled (service-area overlap, deliberate spillover, or a shared upstream resource). A **correctness trap**: a single **global** τ across independent clusters is the **wrong objective** — if one cluster is saturated, a global τ locks onto its worst case and the optimizer goes indifferent to the others. So use **per-cluster objectives** τ_k ; decomposition is a modeling requirement, not just a speed optimization. Tier fallback (metro $\rightarrow$ regional $\rightarrow$ national) is a soft affinity: add $\lambda \sum_{ij} c_{ij} x_{ij}$ with c_{ij} a latency/tier cost — “shave the peak first; among equal-peak solutions, stay local.”

E Appendix: The forecasting method zoo

A **base method** forecasts the residual $R = \log X - FB$ that the exogenous fit leaves; the engine reassembles $\hat{X} = \exp(F_{\text{future}}B + \text{base.forecast}(R))$. Methods differ in how they treat the D channels — **per-channel** (Persistence, Seasonal-naive, AR, Holt-Winters, ARIMA), **multivariate** (Koopman/DMD, one joint operator capturing cross-channel coupling), and **multi-output** (a neural MLP).

Method	Kind	Form / note
Persistence	per-ch.	$\hat{x}_{t+h} = x_t$; the random-walk baseline (skill denominator)
Seasonal-naive	per-ch.	$\hat{x}_{t+h} = x_{t+h-m}$; strong on clean seasonality, brittle under bursts
AR(p)	per-ch.	least-squares on last p lags (default $p = 12$); short lags miss long cycles
Holt-Winters	per-ch.	additive level + trend + season (single seasonality)
Koopman / DMD	multivar.	$z_{t+1} = Az_t$ on a delay-embedded state; rank-truncated, eigenvalues clipped to the unit disk
ARIMA / SARIMAX	per-ch.	$(2, 0, 1)$ [+ seasonal]; $d = 0$ enforced-stationary so it cannot diverge
Neural MLP	multi-out	window $w = 32 \rightarrow$ two 64-ReLU layers $\rightarrow D$ outputs; best endogenous learner, but a fixed window can’t reach weekly/yearly cycles

Table 4: The base-method zoo. All satisfy one `fit` $\rightarrow$ `forecast` interface, so `exog` / `ensemble` / `conformal` wrappers compose over any of them.

F Appendix: Resource-aware objective — formal statement

This extends the base model (Section 2) from the workload proxy to the **true** resource utilization, without leaving the LP class. Reuse the indices (i = class, j = UPF) and data (s_i, t_i) , and define the **aggregate per-UPF workload** (linear in x):

$$S_j(x) = \sum_i x_{ij} s_i, \quad T_j(x) = \sum_i x_{ij} t_i, \quad (11)$$

so $\text{SessLoad}_j = S_j / S_j^{\max}$ and $\text{ThrLoad}_j = T_j / T_j^{\max}$ (the base model’s loads).

Learned per-UPF map. One affine model **per UPF** maps workload to resource use, with coefficients fit by **non-negative least squares** over T observed epochs of the per-UPF gauges (resource rises with load, so coefficients ≥ 0):

$$\widehat{\text{CPU}}_j = c_{0,j}^{\text{cpu}} + a_j^{\text{cpu}} S_j + b_j^{\text{cpu}} T_j, \quad (a_j^{\text{cpu}}, b_j^{\text{cpu}}, c_{0,j}^{\text{cpu}}) = \arg \min_{a,b,c_0 \geq 0} \sum_{k=1}^T (\text{CPU}_{j,k} - a S_{j,k} - b T_{j,k} - c_0)^2 \quad (12)$$

and identically for $\widehat{\text{MEM}}_j$ from `nf_memory_mb`. One regression per UPF is what captures **hardware heterogeneity** — a software UPF and a DPU-offloaded one get different b_j^{cpu} (CPU per Mbps). The resource-aware LP (Equation 4) then bounds **both** $\widehat{\text{CPU}}_j / C_j^{\text{cpu}}$ and $\widehat{\text{MEM}}_j / C_j^{\text{mem}}$ by τ — still an LP, now with $2M$ epigraph rows.

Implied UpfOI weights. Matching the fitted CPU utilization to the proxy form $w_{\text{sess}}(S_j/S^{\text{max}}) + w_{\text{thr}}(T_j/T^{\text{max}})$ gives, per UPF, $w_{\text{sess},j} \propto a_j^{\text{cpu}} S^{\text{max}}$ and $w_{\text{thr},j} \propto b_j^{\text{cpu}} T^{\text{max}}$ (normalized to sum 1) – directly comparable to the fixed (0.40, 0.60). This is a diagnostic, not part of the solve; on the demonstrated fleet the throughput weight recovers as ~ 0.88 (software) vs. ~ 0.72 (DPU-offloaded).

Per-slice extension. Partition classes into G groups by slice / 5QI ($g(i)$ = class i 's group); the per-(UPF, group) loads are still linear in x , and the coefficients are indexed by group:

$$S_{j,g}(x) = \sum_{i:g(i)=g} x_{ij} s_i, \quad \widehat{\text{CPU}}_j = c_{0,j} + \sum_{g=1}^G (\alpha_{j,g} S_{j,g} + \beta_{j,g} T_{j,g}). \quad (13)$$

The objective is unchanged (the same min-max LP). The subtlety is **fitting**: `nf_cpu_mc` is reported per UPF, not per slice, so the $(\alpha_{j,g}, \beta_{j,g})$ are inferred by regressing **aggregate** per-UPF CPU on the per-group load vector – identifiable only when the group mixes vary independently. Because the optimizer **exploits** fit error, the fit adds a **ridge penalty on each coefficient's deviation from its group mean**, shrinking toward the pooled per-UPF estimate; the strength is chosen by time-blocked cross-validation, and in the limit ties all groups back to the per-UPF model. Thin or collinear data therefore collapses smoothly to the per-UPF model rather than fitting noise (the graceful-degradation result, Figure 4).

G Appendix: Receding-horizon MPC – full formulation

Each epoch t , MPC optimizes the **availability trajectory** $m_0, m_1, \dots, m_H$ over an H -epoch horizon (m_k = instances available at horizon step k ; $m_0 = M_{\text{now}}$ fixed), applies only the first action, and re-plans (receding horizon):

$$\min_{m_1, \dots, m_H} \underbrace{w_{\text{ov}} \sum_{k=0}^H (\ell_k/m_k - \text{target})_+}_{\text{overload}} + \underbrace{\lambda_i \sum_{k=1}^H m_k}_{\text{capacity}} + \underbrace{\lambda_c \sum_{k=1}^H |m_k - m_{k-1}|}_{\text{switching}}. \quad (14)$$

Symbols and ranges. All sums run over the H forecast steps ($H = 8$ in the studies, $\geq L$); m_0 is fixed, so the capacity and switching sums start at $k = 1$. ℓ_k is the **forecast load** at step k – the balanced-UpfOI numerator $\ell_k = 100(w_{\text{sess}} s_k/S^{\text{max}} + w_{\text{thr}} t_k/T^{\text{max}})$ at the forecast (s_k, t_k) – so balanced UpfOI at availability m_k is ℓ_k/m_k . **target** is the UpfOI **operating point** overload is referenced to (default 50, below the 60 High-Stress threshold, for headroom) – **not** the 0.6 weight and **not** the 60 line. $(\cdot)_+ = \max(\cdot, 0)$; `inv_pos` is the convex reciprocal $1/m$ for $m > 0$, which keeps the overload term convex in m_k .

The three terms trade **overload** against **capacity** ($\lambda_i \sum_k m_k$ minimizes total instance-epochs – the resource bill) and **switching** (λ_c , an L_1 total-variation anti-oscillation cost). **Dynamics**: a request issued now becomes available only after the spin-up latency L , so m_k can exceed committed capacity only for $k \geq L$; scale-in is immediate, one instance/epoch. Each term is convex, so every epoch is a small convex program (CLARABEL); integer M by rounding. **No terminal cost**: there is no explicit penalty/constraint on m_H (the usual MPC stability device); instead the receding re-plan, $H \geq L$, and a **reactive safety floor** (never provision below what the current load needs, so $\text{MPC} \geq \text{reactive}$) keep it well-behaved without a formal stability claim. Greedy predictive is exactly the $\lambda_c = 0$, single-action ($H = L$) special case – MPC strictly generalizes it. The MPC loop is scalar- M control against the closed-form balanced UpfOI; it does **not** run the intra-UPF placement LP, so every policy compared (static / reactive / predictive / MPC) is on the same balanced-load model.

H Appendix: The metro coupled-twin scenario

The operator-scale study (Section 5) is built as follows.

Nodes. Eight metro **zones** on a line (positions 0..7), alternating archetype: even = **business** (few residents ~ 200 , many jobs ~ 1000), odd = **residential** (many residents ~ 1000 , few jobs ~ 200), each with a random 0.7–1.3 spread. Position on the line makes adjacent zones neighbours.

Mobility / OD. A parameter-free **radiation** kernel maps node populations and distances to a commute origin→destination matrix P_{ij} (probability a commuter from i works at j); because business nodes are job-rich, P routes residential commuters onto nearby business nodes. Work-time population is $\text{work}_z = (1 - \varphi)\text{home}_z +$

$\varphi(\text{home} \cdot P)_z$ (commute fraction $\varphi = 0.6$), and presence is $(1 - a(t))\text{home}_z + a(t)\text{work}_z$ with a midday at-work bell $a(t)$ — so the anti-phase **emerges** from mobility, presence conserved (Figure 5, left).

Eligibility. With $N = 8,000$ classes (1,000/zone) and $M = 48$ UPFs (6/zone), A is 8000×48 , but its structure is an 8×8 zone-adjacency pattern (Figure 5). The **independent** mask A_{block} is block-diagonal (a class eligible only on its own zone’s 6 UPFs); the **coupled twin** $A_{\text{coupled}} = A_{\text{block}} \vee (|\text{zone}_i - \text{zone}_j| = 1)$ is tridiagonal (also the two neighbour zones’ UPFs — interior zones reach $3 \times 6 = 18$, boundary zones 12). Because zones alternate archetype, **every spillover edge joins a business zone to a residential one** — the anti-phased pair — which is exactly why pooling pays.

Preference not used. The coupled solve uses **hard eligibility only** (no `pref_cost`): serving a class on a neighbour’s UPF is treated as **equally** preferable to local, to measure the **pure pooling ceiling**. This is precisely the multi-tier/spillover regime where a tier cost c_{ij} **would** earn its keep (Appendix D) — left off here to isolate the pooling effect.

UPF allocation. Uniform and fixed: **6 UPFs per zone**, 48 total, assigned in contiguous blocks. It does **not** change with time — the pooling result is a single business-peak snapshot solve. What **does** vary is each zone’s demand-driven **need** (Figure 17): business zones need the most capacity midday while residential zones trough, and vice versa at night, so a business+residential pair’s combined need is far flatter than either alone — the temporal picture of the pooling win. (Time-varying M is the scale-out lever of Section 4; here M is fixed and the lever is spillover eligibility.)

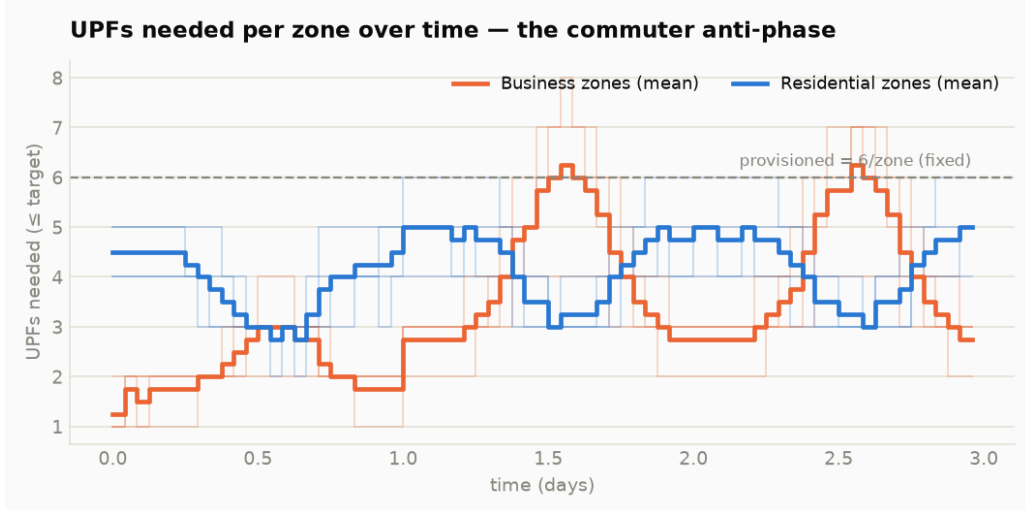


Figure 17: UPFs **needed** per zone over three days (bold = per-archetype mean, faint = individual zones). Business zones (orange) peak midday, residential (blue) off-hours — the commuter anti-phase. The provisioned count is a fixed 6/zone; the gap between **need** and this line is what a dynamic per-zone scale-out, or the coupled pool, absorbs.

I Supplementary material: deployment interfaces and parameters

Integration boundaries. Two typed seams isolate the optimizer/forecaster from the network, so integrating with a real (or externally-emulated) 5G core means implementing an **adapter** on each side — nothing in the core changes.

- **Input** — `FlowSource.sample()` → `FlowSample`: every source yields the same typed per-epoch record (per-class s_i, t_i , the class taxonomy, per-UPF capacities + eligibility). Three interchangeable implementations: a synthetic source, `PrometheusFlowSource` (scrape a live `/metrics`), and `FileFlowSource` (replay archived history for training). The metric → optimizer mapping is a machine-readable contract (below).
- **Output** — `Advisory` → `ActuatorSink.apply()`: the controller emits one typed `Advisory` per epoch — the mirror of `FlowSample` — carrying the two levers it pulls: **placement** (per-class UPF selection weights) and **scale_out/scale_in**, plus a **churn cap**. `SmfActuator` is the emulated sink; a live `SmfEmsAdapter` (push N4/PCF selection weights + call the EMS scale API) is the remaining integration piece — everything above it runs end-to-end today.

Optimizer quantity	Kind	Source series / derivation
throughput t_i	counter	N3/N6 packet count (UL+DL); <code>rate()</code> — the dominant UPF cost
sessions s_i	gauge	Total active sessions per UPF (per-class needs DNN/S-NSSAI labels)
drop rate	counter	N3/N6 drop count — feeds Downlink-Drop / Forwarding-Efficiency SLA proxies
control churn	counter	PFCP message / error count — early warning before throughput saturates
cpu, mem	gauge	<code>nf_cpu_mc</code> / <code>nf_memory_mb</code> — ground truth for the resource-aware objective (Section F)

Table 5: The metrics contract (METRIC_CONTRACT): which Prometheus series feeds each optimizer quantity, and whether it is a counter (needs `rate()`) or a gauge. PrometheusFlowSource scrapes the same schema the round-trip emits, so integration is tested offline before touching a live core.

Scale-out and MPC parameters (the demonstrated scenario, Section 4):

Symbol	Meaning	Value
t_{epoch} (<code>dt_s</code>)	epoch length (re-solve cadence)	600 s
<code>days · epochs</code>	run horizon	$6 \cdot 144/\text{day} = 864$
M_0/M_{max}	initial / max instances	2 / 12
<code>threshold / target</code>	High-Stress line / sizing operating point	60 / 48
L (latency)	spin-up latency (epochs until a requested instance is available)	6 = 60 min
H (horizon)	forecast look-ahead ($H \geq L$)	8 = 80 min
<code>min_train</code>	epochs before the forecaster activates (predictive warm-up)	144 (1 day)
<code>surge</code>	daily event: start · duration · magnitude	16:48 · ~ 1.9 h · ×2.3

Table 6: The scale-out / MPC scenario parameters. M is sized from a closed-form balanced-UpfOI model (`m_needed`); the intra-UPF LP is the orthogonal lever, invoked only in the combined loop.

Acknowledgements

We thank the funding support of the Ministry of Electronics and Information Technology, Government of India, under the MPPLAB project, and collaborators from C-DOT and C-DAC. This work was done with the aid of Claude Opus 4.8.

References

5G standards (3GPP)

- TS 23.501 — System architecture for the 5G System (AMF, SMF, UPF, PDU session, QoS flow, PSA-UPF).
- TS 23.502 — Procedures for the 5G System (PDU session establishment, UPF selection, N4).
- TS 23.503 — Policy and charging control framework (5QI / QoS).
- TS 29.244 — Interface between control-plane and user-plane nodes (PFCP, N4).
- TS 28.552 / TS 28.554 — 5G performance measurements and end-to-end KPIs.

Optimization: theory & algorithms

- S. Boyd and L. Vandenberghe, **Convex Optimization**, Cambridge Univ. Press, 2004.
- S. Mehrotra, “On the implementation of a primal-dual interior point method,” **SIAM J. Optimization** 2(4):575–601, 1992.
- J. Nocedal and S. J. Wright, **Numerical Optimization**, 2nd ed., Springer, 2006.
- D. P. Bertsekas, **Network Optimization: Continuous and Discrete Models**, Athena Scientific, 1998.
- B. Fortz and M. Thorup, “Internet traffic engineering by optimizing OSPF weights,” **IEEE INFOCOM**, 2000.

Solvers & modeling

- S. Diamond and S. Boyd, “CVXPY: A Python-embedded modeling language for convex optimization,” **JMLR** 17(83):1–5, 2016.
- Q. Huangfu and J. A. J. Hall, “Parallelizing the dual revised simplex method” (HiGHS), **Math. Programming Computation** 10(1):119–142, 2018.
- Clarabel — conic interior-point solver (CVXPY’s native backend); CuClarabel — GPU port.
- NVIDIA cuOpt — GPU LP (barrier / PDL); NVIDIA cuDSS — GPU sparse direct KKT solver.

Forecasting & uncertainty

- L. Kleinrock, **Queueing Systems, Vol. 1**, Wiley, 1975 (teletraffic, $M/M/\infty$, Little’s law).
- S. J. Taylor and B. Letham, “Forecasting at scale” (Prophet), **The American Statistician** 72(1):37–45, 2018.
- P. R. Winters, “Forecasting sales by exponentially weighted moving averages,” **Management Science** 6(3):324–342, 1960.
- G. E. P. Box and G. M. Jenkins, **Time Series Analysis: Forecasting and Control**, 1970.
- P. J. Schmid, “Dynamic mode decomposition of numerical and experimental data,” **J. Fluid Mechanics** 656:5–28, 2010.
- J. N. Kutz, S. L. Brunton, B. W. Brunton, J. L. Proctor, **Dynamic Mode Decomposition**, SIAM, 2016.
- J. M. Bates and C. W. J. Granger, “The combination of forecasts,” **Operational Research Quarterly** 20(4):451–468, 1969.
- V. Vovk, A. Gammernan, G. Shafer, **Algorithmic Learning in a Random World**, Springer, 2005 (conformal prediction).
- I. Gibbs and E. Candès, “Adaptive conformal inference under distribution shift,” **NeurIPS**, 2021.
- R. L. Winkler, “A decision-theoretic approach to interval estimation,” **JASA** 67(337):187–191, 1972.
- T. Gneiting and A. E. Raftery, “Strictly proper scoring rules, prediction, and estimation,” **JASA** 102(477):359–378, 2007.

Human mobility

- H. Barbosa et al., “Human mobility: Models and applications,” **Physics Reports** 734:1–74, 2018.
- F. Simini, M. C. González, A. Maritan, A.-L. Barabási, “A universal model for mobility and migration patterns” (radiation model), **Nature** 484:96–100, 2012.
- M. Schlöpfer et al., “The universal visitation law of human mobility,” **Nature** 593:522–527, 2021.
- C. Song, Z. Qu, N. Blumm, A.-L. Barabási, “Limits of predictability in human mobility,” **Science** 327:1018–1021, 2010.

5G core, observability & sizing

- Prometheus (metrics/TSDB); Grafana (dashboards); Open5GS and free5GC (open-source 5G cores; source of `amf_session`, `nf_cpu_mc`, `nf_memory_mb`).
- TRAI India wireless subscriber data (Dec 2025).
- Jio distributed cloud-native 5G core / UPF (“5G on the Edge”, 2.8 Tbps/cluster).
- Per-UPF instance capacity ~ 100 Gbps (IPLOOK 5GC UPF).
- NVIDIA accelerated & distributed UPF; Intel / Dell / Nokia far-edge UPF deployment.